E-Commerce Project

Installation & Setup Guide

Full-Stack Multi-Language E-Commerce Platform

(English / French / Arabic)

July 4, 2026

Project Overview

This project consists of three applications:
Backend API (Express + MongoDB + Redis)
Admin Dashboard (React-Admin + Vite)
Storefront (Next.js + Tailwind CSS)

Contents

1 Prerequisites

Before starting, make sure you have the following installed or accounts created:

Software	Version	Purpose
Node.js	≥ 18	JavaScript runtime for all apps
MongoDB	7+	Main database
MongoDB Compass	Latest	GUI for database management
Redis (Upstash)	Cloud	Caching, rate limiting, OTP
Git	Latest	Version control (optional)
VS Code	Latest	Code editor (recommended)

Accounts needed:

- Upstash — Free Redis cloud database
- Cloudinary — Image upload and CDN
- Google Cloud Console — Google OAuth credentials
- Stripe — Payment processing
- MongoDB Atlas (optional) — Cloud database

2 Node.js Installation

Node.js is required to run all three applications (backend, dashboard, and storefront).

2.1 Download and Install

1. Visit https://nodejs.org/
2. Download the **LTS** version (18.x or higher — the left button)
3. Run the downloaded `.msi` installer
4. Click **Next** through the wizard (default settings are fine)
5. Make sure **"Add to PATH"** is checked during installation
6. Finish the installation

2.2 Verify Installation

Open a new terminal (Command Prompt or PowerShell) and run:

```
node --version
npm --version
```

Listing 1: Verify Node.js and npm

You should see version numbers (e.g., `v18.17.0` and `10.8.0`).

YouTube Tutorial

How to Install Node.js on Windows 11 — Amit Thinks (Updated 2025)
https://www.youtube.com/watch?v=lt5D2EWZMN0

3 MongoDB Installation

MongoDB is the primary database used by the backend API.

3.1 Option 1: Install MongoDB Locally (Recommended)

1. Go to https://www.mongodb.com/try/download/community
2. Select your operating system (Windows)
3. Choose **MongoDB 7.0+ Community Server**
4. Click **Download**
5. Run the installer (`.msi` file)
6. Choose **"Complete"** setup type
7. **Important:** Check **"Install MongoDB Compass"** during installation
8. Complete the installation
9. MongoDB runs automatically as a Windows service

3.2 Verify MongoDB

Open Command Prompt or PowerShell and run:

```
mongosh
```

Listing 2: Test MongoDB connection

If successful, you will see the MongoDB shell with `test>` prompt. Type `exit` to leave.

YouTube Tutorial

Install MongoDB, MongoDB Compass, and MongoDB Shell (mongosh) on Windows
https://www.youtube.com/watch?v=jvaBaxlTqU8

3.3 Option 2: MongoDB Atlas (Cloud)

If you prefer not to install MongoDB locally, use MongoDB Atlas (free tier):

1. Go to https://www.mongodb.com/atlas
2. Click **"Try Free"** and sign up
3. Choose the **Free M0 Sandbox** cluster
4. Select a cloud provider and region close to you
5. Click **"Create Cluster"** (takes 1–3 minutes)
6. In **Database Access** tab:

- Click **"Add New Database User"**
- Create a username and password
- Choose **"Read and Write to Any Database"** privileges
- Click **"Add User"**

7. In **Network Access** tab:
   - Click **"Add IP Address"**
   - Click **"Allow Access from Anywhere"** (`0.0.0.0/0`) for development
   - Click **"Confirm"**
8. Click **"Connect"** → **"Connect your application"**
9. Copy the connection string (looks like `mongodb+srv://username:password@cluster.xxxxx.mongod`
10. Replace `<password>` with your user's password
11. Use this string as `MONGODB_URI` in your `.env` file

YouTube Tutorial

How to Get a Free MongoDB Database in the Cloud — Step-by-Step Setup
https://www.youtube.com/watch?v=SFkXQZ3x6zM&t=82s

4 MongoDB Compass

MongoDB Compass is a graphical user interface (GUI) for MongoDB. It lets you view, query, and manage your data visually.

4.1 If Installed with MongoDB

Compass is installed automatically if you checked the box during MongoDB installation.

4.2 Standalone Installation

1. Download from https://www.mongodb.com/products/tools/compass
2. Install and open the application
3. Paste your connection string (e.g., `mongodb://localhost:27017`) and click **"Connect"**
4. You can now browse databases, collections, and documents visually

YouTube Tutorial

Same video as MongoDB installation above:
Install MongoDB, MongoDB Compass, and MongoDB Shell (mongosh) on Windows
https://www.youtube.com/watch?v=jvaBaxlTqU8

5 Redis Setup (Upstash Cloud)

This project uses **Upstash** — a cloud-based Redis service. No local installation is needed.

5.1 Why Upstash Instead of Local Redis?

- No installation or configuration required
- Free tier (10,000 commands per day — enough for development)
- Works anywhere without firewall or networking issues
- Persistent storage with automatic backups

5.2 Create an Upstash Account

1. Go to https://upstash.com/
2. Click **"Start Free"** or **"Sign Up"**
3. Sign up using **GitHub** or **Google** (quickest)
4. You will be redirected to the console: https://console.upstash.com/

5.3 Create a Redis Database

1. In the Upstash console, click **"Create Database"**
2. Give it a name (e.g., `e-commerce-redis`)
3. Choose a region close to you (e.g., **Frankfurt** for Europe, **Ohio** for US East)
4. Leave the default settings
5. Click **"Create"**
6. You will see the database details page

5.4 Get the Connection URL

On the database details page, find and copy the `REDIS_URL` (also called `REST URL`). It looks like:

```
rediss://default:AVM2AAI3ASDASD12345@refined-panda-100748.upstash.io
    :6379
```

Listing 3: Redis connection URL example

This string goes into your `config/.env` file as:

```
REDIS_URL=rediss://default:YOUR_TOKEN@your-database.upstash.io:6379
```

Listing 4: .env entry for Redis

5.5 How Redis Is Used in This Project

- **IP Blocklist** — Blocks IP addresses that send too many malicious requests
- **Rate Limiting** — Limits how fast users can send requests (prevent spam)
- **OTP Storage** — Stores email verification codes temporarily
- **Session Management** — Manages user sessions (if session-based auth is used)

Redis is Optional

If Redis is unavailable or you skip setup, the application still works normally. Features like rate limiting and IP blocking are gracefully skipped with a warning log.

YouTube Tutorial

Upstash Redis Tutorial — Get Started with Serverless Redis
https://www.youtube.com/watch?v=lK2DmL7t5R8
Direct links:

- Upstash website: https://upstash.com/
- Upstash console: https://console.upstash.com/redis/

6 Email Setup (Gmail App Password)

The project uses Gmail to send:

- Email verification links after signup
- Password reset links

6.1 Steps to Get Gmail App Password

1. **Turn on 2-Step Verification** (required for App Passwords)
   (a) Go to https://myaccount.google.com/security
   (b) Under **"How you sign in to Google"**, click **"2-Step Verification"**
   (c) Follow the steps to enable it (requires a phone number)

2. **Generate an App Password**
   (a) Go to https://myaccount.google.com/apppasswords
   (b) (If asked, sign in to your Google account)
   (c) Under **"Select app"**, choose **Mail**
   (d) Under **"Select device"**, choose **Other (Custom name)**
   (e) Type a name like `"E-Commerce Backend"` or `"My App"`
   (f) Click **Generate**
   (g) Google will show a **16-character password** like:

Example App Password

`wceh rtqu flmu crle`

   (h) **Copy this password immediately** — you will not see it again

6.2 Add to .env

```
EMAIL = your-email@gmail.com
PASSWORD = wcehrtquflmucrle   # Your 16-character app password (no
    spaces)
```

Listing 5: Email configuration in config/.env

Important Notes

- You **must** use the App Password, NOT your regular Gmail password
- Remove spaces from the generated password
- If you change your Google account password, App Passwords are reset
- If you don't see the App Passwords option, make sure 2-Step Verification is enabled

7 Google OAuth Setup

Google OAuth allows users to sign up and log in using their Google account.

7.1 Create a Google Cloud Project

1. Go to https://console.cloud.google.com/
2. Sign in with your Google account
3. Click the project dropdown at the top (next to "Google Cloud")
4. Click **"New Project"**
5. Give it a name (e.g., `"E-Commerce OAuth"`)
6. Click **"Create"**
7. Make sure your new project is selected in the top bar

7.2 Configure OAuth Consent Screen

1. Go to **APIs & Services** → **OAuth consent screen**
2. Choose **"External"** (for testing with any Google account)
3. Click **"Create"**
4. Fill in:
   - **App name**: `E-Commerce`
   - **User support email**: Your email
   - **Developer contact information**: Your email
5. Click **"Save and Continue"**
6. **Scopes**: Click **"Add or Remove Scopes"**, check:
   - `.../auth/userinfo.email`
   - `.../auth/userinfo.profile`
7. Click **"Update"**, then **"Save and Continue"**
8. **Test Users**: Click **"Add Users"**, add your email
9. Click **"Save and Continue"**, then **"Back to Dashboard"**

7.3 Create OAuth 2.0 Credentials

1. Go to **APIs & Services** → **Credentials**
2. Click **"+ Create Credentials"** → **"OAuth client ID"**
3. Choose **"Web application"**
4. **Name**: `Web Client`
5. Under **"Authorized JavaScript Origins"**:
   - Click **"Add URI"**
   - Add: `http://localhost:3007`
6. Under **"Authorized Redirect URIs"**:
   - Click **"Add URI"**
   - Add: `http://localhost:3007/v1/auth/google/signup/callback`
   - Click **"Add URI"** again
   - Add: `http://localhost:3007/v1/auth/google/login/callback`
7. Click **"Create"**

7.4 Copy Credentials

After creating, a popup shows:

- **Client ID**: `244052021863-xxxxx.apps.googleusercontent.com`
- **Client Secret**: `GOCSPX-xxxxxxxxxxx`

Save These Values!

Copy both values and add them to your `config/.env`:

```
GOOGLE_CLIENT_ID = 244052021863-xxxxx.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET = GOCSPX-xxxxxxxxxxx

```

YouTube Tutorial

Google OAuth 2.0 Setup Tutorial
https://www.youtube.com/watch?v=Q0a0594tOrc&t=8s

8 Cloudinary Setup (Image Uploads)

Cloudinary is used to upload, store, and transform product images.

8.1 Create a Cloudinary Account

1. Go to https://cloudinary.com/
2. Click **"Sign Up for Free"**
3. You can sign up with Google, GitHub, or email
4. Verify your email address
5. You will be redirected to the Cloudinary Dashboard

8.2 Find Your Credentials

On the Cloudinary Dashboard, find these three values:

1. **Cloud Name** — Looks like `dwld0gbaj`
2. **API Key** — A number like `884793678717992`
3. **API Secret** — A string like `hZy1DbqQiwBJHbfKNDBjyaEP-Wo`

Add to .env

```
Image_TYPE = CLOUDINARY
CLOUDINARY_NAME = dwld0gbaj
CLOUDINARY_KEY = 884793678717992
CLOUDINARY_SECRET = hZy1DbqQiwBJHbfKNDBjyaEP-Wo

```

8.3 Optional: Fallback to Server Storage

If you don't want to use Cloudinary, you can set:

```
Image_TYPE = SERVER
SERVER = http://localhost:3007
```

This will store images on your server's local filesystem instead.

9 Stripe Setup (Payments)

Stripe handles payment processing in the storefront.

9.1 Create a Stripe Account

1. Go to https://stripe.com/
2. Click **"Start now"** or **"Sign up"**
3. Enter your email and create a password
4. Complete the onboarding (you can skip if asked for business details)

9.2 Get API Keys

1. Log in to the Stripe Dashboard: https://dashboard.stripe.com/
2. Make sure you are in **"Test mode"** (toggle in the left sidebar)
3. Go to **Developers** → **API Keys**
4. You will find:
   - **Publishable Key**: `pk_test_51TiY5yATJO...`
   - **Secret Key**: `sk_test_51TiY5yATJO...`
5. **Copy both keys**

9.3 Add to .env

```
Publishable_key_Stripe = pk_test_51TiY5yATJO...
Secret_key_Stripe = sk_test_51TiY5yATJO...
```

Listing 6: Stripe configuration in config/.env

Test Mode

Test mode keys start with `pk_test_` and `sk_test_`.
You can use test credit card number `4242 4242 4242 4242` for payments.
For production, switch to **Live mode** and use live keys (start with `pk_live_` / `sk_live_`).

10 Full Project Setup

10.1 Clone the Repository

```
git clone <repository-url>
cd e-commerce
```

Listing 7: Clone the project

10.2 Project Structure

The project has three main folders:

```
e-commerce/
  |-- e-commerce-back/     # Backend API (Express + MongoDB + Redis)
  |-- e-commerce-Dash/     # Admin Dashboard (React-Admin + Vite)
  |-- e-commerce-Next/     # Storefront (Next.js + Tailwind CSS)
  |-- installation.tex      # This installation guide
  |-- README.md             # Project documentation
```

Listing 8: Project folder structure

11 Backend Setup (e-commerce-back)

11.1 Install Dependencies

```
cd e-commerce-back
npm install
```

Listing 9: Install backend packages

11.2 Configure Environment Variables

1. Copy the example file:

```
cp config/.env.example config/.env
```

2. Edit `config/.env` with all the credentials you collected:

```
PORT = 3007
EMAIL = your-email@gmail.com
PASSWORD = your-16-char-app-password
HASH = 10
BASE_URL = http://localhost:3007
SIGNATURE_ADMIN = signatureAdmin
SIGNATURE_USER = signatureUser
SIGNATURE_SUPER_ADMIN = signatureSuperAdmin
ACCESS_TOKEN = 30m
REFRESH_TOKEN = 1y
VERIFY_SIGNATURE_MY = my
MONGODB_URI = mongodb://localhost:27017/e-commerce
GOOGLE_CLIENT_ID = your-client-id.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET = GOCSPX-xxxxxxxxxxx
REDIS_URL = rediss://default:xxxxx@your-database.upstash.io:6379
Image_TYPE = CLOUDINARY
CLOUDINARY_NAME = your-cloud-name
CLOUDINARY_KEY = your-api-key
CLOUDINARY_SECRET = your-api-secret
SHIPPING_COST_TYPE = fixed
SHIPPING_COST_VALUE = 50
CURRENCY = EGP
Publishable_key_Stripe = pk_test_xxxxxxxxxxxx
Secret_key_Stripe = sk_test_xxxxxxxxxxxx
```

Listing 10: Complete config/.env example

11.3 Run the Backend

```
npm run dev
```

Listing 11: Start backend server

Server Running

You should see:

```
Server running on port 3007
database connected
```

11.4 API Endpoints

Once running, the API is available at `http://localhost:3007/v1`

- `POST /v1/auth/signup` — Register a new user
- `POST /v1/auth/login` — Login
- `GET /v1/products` — List active products
- `GET /v1/category` — List categories
- `GET /v1/subcategory` — List subcategories
- And more...

12 Dashboard Setup (e-commerce-Dash)

React-Admin dashboard for managing products, categories, users, and orders.

12.1 Install Dependencies

```
cd e-commerce-Dash
npm install
```

Listing 12: Install dashboard packages

12.2 Configure Environment Variables

Create a file named `.env` inside `e-commerce-Dash/`:

```
VITE_API_URL=http://localhost:3007/v1
```

Listing 13: .env for dashboard

12.3 Run the Dashboard

```
npm run dev
```

Listing 14: Start dashboard

Dashboard runs at `http://localhost:8000`

12.4 Dashboard Features

- **Categories** — Create, edit, list categories
- **Subcategories** — Create, edit, group delete/restore
- **Products** — Create, edit, list with variants management
- **Orders** — View and manage customer orders
- **Users** — View users, create admin accounts (super admin only)
- **Analytics** — Revenue chart, order statistics
- **Multi-language** — Switch between English, French, Arabic
- **Themes** — Multiple themes (Soft, B&W, Nano, etc.)

13 Storefront Setup (e-commerce-Next)

Next.js storefront for customers to browse and purchase products.

13.1 Install Dependencies

```
cd e-commerce-Next
npm install
```

Listing 15: Install storefront packages

13.2 Configure Environment Variables

Create a file named `.env` inside `e-commerce-Next/`:

```
BACK_URL=http://localhost:3007
NEXT_PUBLIC_SHIPPING_COST_TYPE=fixed
NEXT_PUBLIC_SHIPPING_COST_VALUE=50
```

Listing 16: .env for storefront

13.3 Run the Storefront

```
npm run dev
```

Listing 17: Start storefront

Storefront runs at http://localhost:3000

13.4 Storefront Features

- **Home Page** — Featured products, categories
- **Products Page** — Browse with filters, sort, search
- **Product Detail** — View details, select variants, add to cart
- **Cart** — Manage items, proceed to checkout
- **Checkout** — Stripe payment integration
- **User Account** — Login, signup, profile, orders
- **Wishlist** — Save favorite products
- **Multi-language** — English, French, Arabic (RTL support)

14 Running All Applications

Open **three separate terminals** and run:

```
cd e-commerce-back
npm run dev
```

Listing 18: Terminal 1 - Backend

```
cd e-commerce-Dash
npm run dev
```

Listing 19: Terminal 2 - Dashboard

```
cd e-commerce-Next
npm run dev
```

Listing 20: Terminal 3 - Storefront

14.1 Access Points Summary

Application	URL	Port
Backend API	http://localhost:3007	3007
Admin Dashboard	http://localhost:8000	8000
Storefront	http://localhost:3000	3000

15 Useful Links

15.1 Download Links

Software	Link
Node.js	https://nodejs.org/
MongoDB Community	https://www.mongodb.com/try/download/community
MongoDB Compass	https://www.mongodb.com/products/tools/compass
MongoDB Atlas	https://www.mongodb.com/atlas
Visual Studio Code	https://code.visualstudio.com/
Git	https://git-scm.com/
Postman	https://www.postman.com/

15.2 Service Links

Service	Link
Upstash (Redis)	https://upstash.com/
Upstash Console	https://console.upstash.com/redis/
Cloudinary	https://cloudinary.com/
Google Cloud Console	https://console.cloud.google.com/
Google App Passwords	https://myaccount.google.com/apppasswords
Stripe Dashboard	https://dashboard.stripe.com/

15.3 YouTube Tutorials

Topic	Link
Node.js Install (Windows 11)	https://www.youtube.com/watch?v=lt5D2EWZMN0
MongoDB + Compass Install	https://www.youtube.com/watch?v=jvaBaxlTqU8
MongoDB Atlas (Free Cloud DB)	https://www.youtube.com/watch?v=SFkXQZ3x6zM&t=82s
Upstash Redis Setup	https://www.youtube.com/watch?v=lK2DmL7t5R8
Google OAuth 2.0 Setup	https://www.youtube.com/watch?v=Q0a0594tOrc&t=8s
Git Install and Setup	https://www.youtube.com/watch?v=8Dd7KR9KePk
VS Code Setup	https://www.youtube.com/watch?v=UWQe6zCjQ84

Thank You!

You now have everything needed to run the project.
Good luck with your E-Commerce platform!

For issues, check the project's README.md or create a GitHub issue.